

CS432: Databases

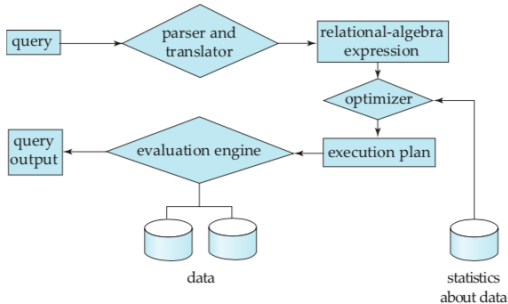
Query Processing and Optimizatoin

Instructor
Yogesh K. Meena

Lecture no.
20

CSE, IIT Gandhinagar
April 08, 2026

Steps in query processing



- Parsing and translation
- Optimization
- Evaluation

Evaluating different candidates

An example SQL query

```
select salary  
from instructor  
where salary < 75000;
```

Evaluating different candidates

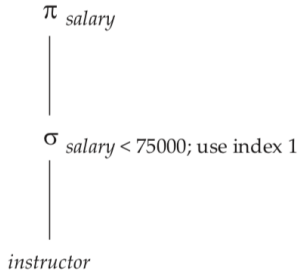
An example SQL query

```
select salary  
from instructor  
where salary < 75000;
```

Possible relational algebra expressions

$$\sigma_{\text{salary} < 75000} (\Pi_{\text{salary}} (\text{instructor}))$$
$$\Pi_{\text{salary}} (\sigma_{\text{salary} < 75000} (\text{instructor}))$$

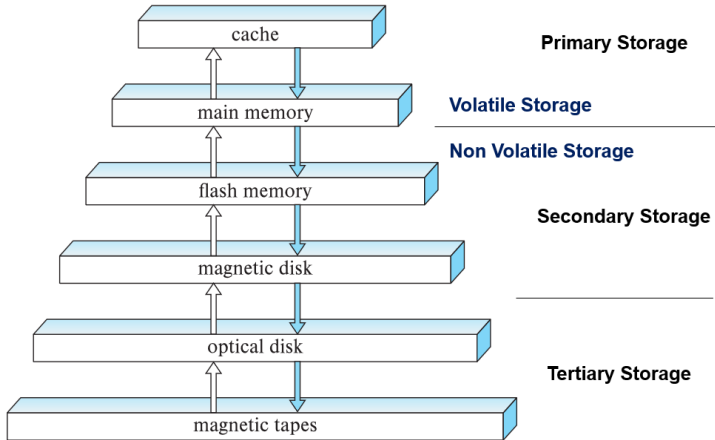
A query-evaluation plan



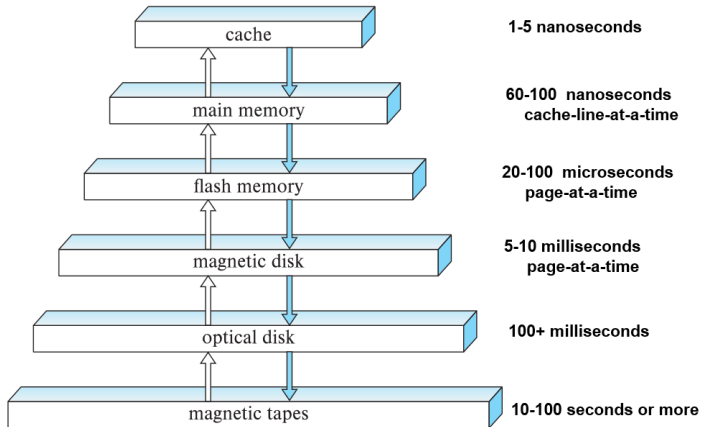
Expressions annotated with instructions specifying how to evaluate each operation. ★ ★

Different evaluation plans have different costs

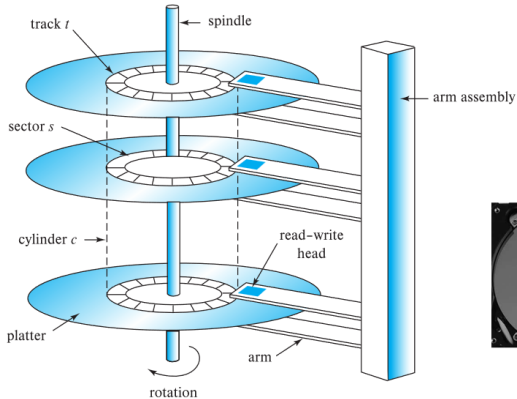
Physical Storage Systems: Storage Hierarchy



Storage Hierarchy: Access Time



Magnetic Hard Disk Mechanism



Schematic diagram of magnetic disk drive

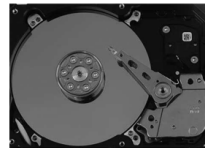


Photo of magnetic disk drive

Measures of Query Cost

- Disk accesses
- CPU time to execute a query
- A distributed or parallel database system
- The cost of communication

The disk activity dominates the total time to execute a query

Measures of Query Cost

- Disk accesses
- CPU time to execute a query
- A distributed or parallel database system
- The cost of communication

The disk activity dominates the total time to execute a query

Estimating cost from disk accesses

t_T seconds = transfer a block of data

t_S seconds = block-access/seek time

An operation that transfers b blocks and performs S seeks takes:

$$b * t_T + S * t_S$$

Why not consider the Response time?

Measures of Query Cost

- Disk accesses
- CPU time to execute a query
- A distributed or parallel database system
- The cost of communication

The disk activity dominates the total time to execute a query

Estimating cost from disk accesses

t_T seconds = transfer a block of data

t_S seconds = block-access/seek time

An operation that transfers b blocks and performs S seeks takes:

$$b * t_T + S * t_S$$

Why not consider the Response time?

We ignore CPU time for simplicity - real system do take it into account.

Selection Operation

Linear Search

The system scans each file block and tests all records to see whether they satisfy the selection condition.

$$\text{Cost} = t_S + b_r * t_T$$

b_r denotes the number of blocks in the file. One initial seek plus b_r block transfers.

Indexes

Primary Index: Records in an index to be read in an order that corresponds to the physical order in the file.

Secondary Index: An index that is not a primary index

Primary index, B^+ -tree Index, equality on key

$$\text{Cost} = (h_i + 1) * (t_S + t_T)$$

h_i = height of B^+ -tree.

Indexes

Primary Index: Records in an index to be read in an order that corresponds to the physical order in the file.

Secondary Index: An index that is not a primary index

Primary index, B^+ -tree Index, equality on key

$$\text{Cost} = (h_i + 1) * (t_s + t_T)$$

h_i = height of B^+ -tree.

Primary index, B^+ -tree Index, equality on nonkey

b = number of blocks containing records with the specified search key

Indexes

Primary Index: Records in an index to be read in an order that corresponds to the physical order in the file.

Secondary Index: An index that is not a primary index

Primary index, B^+ -tree Index, equality on key

$$\text{Cost} = (h_i + 1) * (t_s + t_T)$$

h_i = height of B^+ -tree.

Primary index, B^+ -tree Index, equality on nonkey

b = number of blocks containing records with the specified search key

$$\text{Cost} = h_i * (t_s + t_T) + t_s + b * t_T$$

Indexes

Secondary index, B^+ -tree Index, equality on key

$$\text{Cost} = (h_i + 1) * (t_s + t_T)$$

h_i = height of B^+ -tree.

Indexes

Secondary index, B^+ -tree Index, equality on key

$$\text{Cost} = (h_i + 1) * (t_s + t_T)$$

h_i = height of B^+ -tree.

Secondary index, B^+ -tree Index, equality on nonkey

n = number of records fetched

Indexes

Secondary index, B^+ -tree Index, equality on key

$$\text{Cost} = (h_i + 1) * (t_S + t_T)$$

h_i = height of B^+ -tree.

Secondary index, B^+ -tree Index, equality on nonkey

n = number of records fetched

$$\text{Cost} = (h_i + n) * (t_S + t_T)$$

Indexes

Primary index, B^+ -tree Index, Comparison

$$\text{Cost} = h_i * (t_S + t_T) + t_S + b * t_T$$

h_i = height of B^+ -tree.

Secondary index, B^+ -tree Index, Comparison

$$\text{Cost} = (h_i + n) * (t_S + t_T)$$

Complex Selections

- Conjunction: $\sigma_{\theta_1 \wedge \theta_2 \wedge \theta_3 \dots \wedge \theta_n}(r)$
- Disjunction: $\sigma_{\theta_1 \vee \theta_2 \vee \theta_3 \dots \vee \theta_n}(r)$
- Negation: $\sigma_{\neg \theta}(r)$



Complex Selections

Conjunctive selection using one index

- Choose one attribute in conjunction, fetch all matching records in buffer.
- In buffer, check whether or not each retrieved record satisfies the remaining simple conditions.

Complex Selections

Conjunctive selection using one index

- Choose one attribute in conjunction, fetch all matching records in buffer.
- In buffer, check whether or not each retrieved record satisfies the remaining simple conditions.

Conjunctive selection using composite index

Search the index directly if:

- If the selection specifies an equality condition on two or more attributes and
- a composite index exists on these combined attribute.

The rest operations are left as an exercise!

Sorting: External Sort-Merge Algorithm

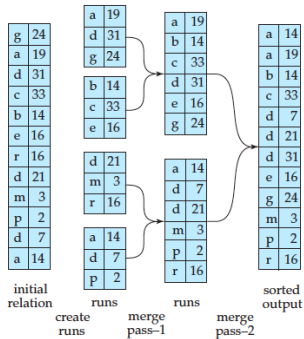
Step 1: Create a number of sorted runs

```
 $i = 0;$   
repeat  
    read  $M$  blocks of the relation, or the rest of the relation,  
        whichever is smaller;  
    sort the in-memory part of the relation;  
    write the sorted data to run file  $R_i$ ;  
     $i = i + 1$ ;  
until the end of the relation
```

Step 2: Merge the sorted runs.

```
read one block of each of the  $N$  files  $R_i$  into a buffer block in memory;  
repeat  
    choose the first tuple (in sort order) among all buffer blocks;  
    write the tuple to the output, and delete it from the buffer block;  
    if the buffer block of any run  $R_i$  is empty and not end-of-file( $R_i$ )  
        then read the next block of  $R_i$  into the buffer block;  
until all input buffer blocks are empty
```

Sorting: External Sort-Merge Algorithm



Sorting

- External Sort-Merge Algorithm
 - Create a number of sorted runs.
 - Merge the sorted runs.
- Total Block Transfers

$$b_r(2\lceil \log_{M-1}(b_r/M) \rceil + 1)$$

No writing of the final result to disk.

Sorting

- External Sort-Merge Algorithm
 - Create a number of sorted runs.
 - Merge the sorted runs.
- Total Block Transfers

$$b_r(2\lceil \log_{M-1}(b_r/M) \rceil + 1)$$

No writing of the final result to disk.

- Total Disk Seeks

$$2\lceil b_r/M \rceil + \lceil b_r/b_b \rceil (2\lceil \log_{M-1}(b_r/M) \rceil - 1)$$

Here, b_b buffer blocks are allocated to each run.

No writing of the final result to disk.

Join Operation

- Cost estimation

- Number of records of student: $n_{student}$
- Number of blocks of student: $b_{student}$
- Number of records of takes: n_{takes}
- Number of blocks of takes: b_{takes}

Compute: $student \bowtie_{\theta} takes$



Join Operation

- Cost estimation
 - Number of records of student: $n_{student}$
 - Number of blocks of student: $b_{student}$
 - Number of records of takes: n_{takes}
 - Number of blocks of takes: b_{takes}

Compute: **student** $\bowtie_{\theta}$ **takes**

Nested-Loop Join algorithm for Theta join

```
for each tuple  $t_r$  in  $r$  do begin
  for each tuple  $t_s$  in  $s$  do begin
    test pair  $(t_r, t_s)$  to see if they satisfy the join condition  $\theta$ 
    if they do, add  $t_r \cdot t_s$  to the result;
  end
end
```

Relation r is called the **outer** relation and relation s the **inner** relation.

Nested-Loop Join algorithm for Theta join

- Worst case (only one block can fit into the memory)

Nested-Loop Join algorithm for Theta join

- Worst case (only one block can fit into the memory)
 - Total block transfers: $n_r * b_s + b_r$
 - Total seeks: $n_r + b_r$
- Best case (all blocks can fit into the memory)

Nested-Loop Join algorithm for Theta join

- Worst case (only one block can fit into the memory)
 - Total block transfers: $n_r * b_s + b_r$
 - Total seeks: $n_r + b_r$
- Best case (all blocks can fit into the memory)
 - Total block transfers: $b_s + b_r$
 - Total seeks: 2
- If only s is small enough to fit in the main memory

Nested-Loop Join algorithm for Theta join

- Worst case (only one block can fit into the memory)
 - Total block transfers: $n_r * b_s + b_r$
 - Total seeks: $n_r + b_r$
- Best case (all blocks can fit into the memory)
 - Total block transfers: $b_s + b_r$
 - Total seeks: 2
- If only s is small enough to fit in the main memory
 - Total block transfers: $b_s + b_r$
 - Total seeks: 2

Block Nested-Loop Join

```
for each block  $B_r$  of  $r$  do begin
  for each block  $B_s$  of  $s$  do begin
    for each tuple  $t_r$  in  $B_r$  do begin
      for each tuple  $t_s$  in  $B_s$  do begin
        test pair  $(t_r, t_s)$  to see if they satisfy the join condition
        if they do, add  $t_r \cdot t_s$  to the result;
      end
    end
  end
end
end
```

Block Nested-Loop Join

- Worst case (only one block can fit into the memory)
 - Total block transfers: $b_r * b_s + b_r$
 - Total seeks: $2 * b_r$
- Best case (all blocks can fit into the memory)
 - Total block transfers: $b_s + b_r$
 - Total seeks: 2

Improving performances of the nested-loop

- If the join attributes forms a **key** on the inner relation, the inner loop can terminate as soon as the first match is found.
- In the block nested-loop algorithm, instead of using disk blocks as the blocking unit for the outer relation, we can use the **biggest size that can fit in memory**, while leaving enough space for the buffers of the inner relation and the output.
- If an **index** is available on the inner loop's join attribute, we can replace file scans with more efficient index lookups.

Indexed Nested-Loop Join

Index lookups replaces file scans in nested-loop join for inner loop's attribute.

- For each tuple t_r in the outer relation r ,
 - the index is used to look up tuples in s that will satisfy the join condition with tuple t_r

Worst case (only one block of r and one block of index can fit into the memory)

- Total access to r blocks: b_r
- Time cost of the join: $b_r(t_T + t_S) + n_r * c$

c is the cost of a single selection on s using the join condition.

Other Operations

Duplicate Elimination

- Eliminate duplicates by sorting.
- Remove duplicates before the run is written to disk, thereby reducing the number of block transfers.
- remove rest of the duplicates during merging.

Duplicate elimination is as costly as sorting of the relation.

Other Operations

Duplicate Elimination

- Eliminate duplicates by sorting.
- Remove duplicates before the run is written to disk, thereby reducing the number of block transfers.
- remove rest of the duplicates during merging.

Duplicate elimination is as costly as sorting of the relation.

Projection

- Perform projection on each tuple.
- Remove duplicates as per duplicate elimination process.

Duplicate elimination is not required if projection list include a key of the relation.

Other Operations

Set Operations

- Sort both relations
- Scan once through each of the sorted relations to produce the result

$r \cup s$: Concurrently scan both relations but keep only a single copy of tuples

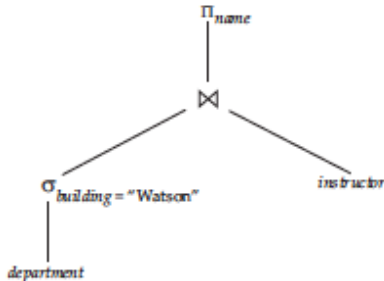
$r \cap s$: Concurrently scan both relations, keep only one of the matching tuples

$r - s$: Retain tuples in r if they are absent in s

- Total block transfers: $b_r + b_s$
- Total block seeks: $b_r + b_s$

Evaluation: Materialization

- Start from the lowest-level operations in the expression and move upwards.
- Store the results in temporary relations.



Query Optimization: Equivalent expressions

instructor(ID, name, dept_name, salary)
teaches(ID, course_id, sec_id, semester, year)
course(course_id, title, dept_name, credits)

Find the names of all instructors in the Music department together with the course title of all the courses that the instructors teach.

Query Optimization: Equivalent expressions

instructor(ID, name, dept_name, salary)
teaches(ID, course_id, sec_id, semester, year)
course(course_id, title, dept_name, credits)

Find the names of all instructors in the Music department together with the course title of all the courses that the instructors teach.

$\Pi_{name, title} (\sigma_{dept_name = \text{"Music"}} (instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$

Is above expression optimized?

Equivalent expressions

Find the names of all instructors in the Music department together with the course title of all the courses that the instructors teach.

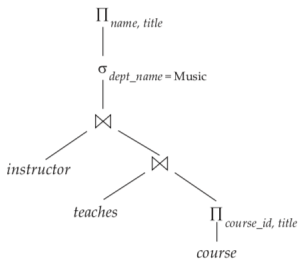
$$\Pi_{name, title} ((\sigma_{dept_name = \text{"Music"}} (instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$$

Equivalent expressions

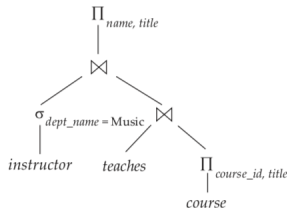
Find the names of all instructors in the Music department together with the course title of all the courses that the instructors teach.

$$\Pi_{name, title} ((\sigma_{dept_name = \text{"Music"}} (instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$$

Expression trees



(a) Initial expression tree



(b) Transformed expression tree

Generating query-evaluation plans

- Generating logically equivalent expressions.
- Annotating expressions to generate alternative query-evaluation plans.
- Cost estimation and selecting the one whose estimated cost is the least.

Above operations are interleaved in the **query optimizer**

Equivalent relational-algebra expressions

Two relational-algebra expressions are said to be equivalent if, on every **legal database** instance, the two expressions generate the **same set of tuples**.

Equivalent relational-algebra expressions

Two relational-algebra expressions are said to be equivalent if, on every **legal database** instance, the two expressions generate the **same set of tuples**.

- **Legal:** All the integrity constraints are satisfied

Equivalent relational-algebra expressions

Two relational-algebra expressions are said to be equivalent if, on every **legal database** instance, the two expressions generate the **same set of tuples**.

- **Legal:** All the integrity constraints are satisfied
- **Ordering** of the tuples is irrelevant

Acknowledgments/Contributions

- Some of the images utilized in these slides are subject to copyright - Abraham Silberschatz, Henry Korth, and S. Sudarshan. Database System Concepts. 6th Edition, McGraw-Hill Education
- Contributor 2024-25, 2025-26 - Yogesh K. Meena, IIT Gandhinagar